

```

import struct
import random
import time
from collections import deque

# =====
# SOVEREIGN PIT-WALL: ZERO-LATENCY AI TELEMETRY
# Architect: Master Blueprint | Bio-Mechanical Equilibrium
# =====

# 1. The Architect's Locked 6-Pillar Weights (Must equal 1.00)
WEIGHTS = {
    'tyres': 0.25,          # 25% - Structural blowout risk
    'driver_fatigue': 0.25, # 25% - Cognitive degradation (Equal to tires)
    'thermal': 0.20,        # 20% - Engine/Coolant operating windows
    'hydraulics': 0.15,     # 15% - Steering/Gearbox pressure
    'brakes': 0.10,        # 10% - Carbon friction wear
    'fuel_system': 0.05     # 5% - Fuel pump pressure
}

# 2. The Dual-Trigger Safety Thresholds
# Critical Holistic Degradation Index (HDI), ranging 0-100, where 0 is "perfect" state and 100 is "dead" state.
CRITICAL_HDI_THRESHOLD = 70.0 # Cascading system failure
REDLINE_VETO = 85.0          # Single-parameter kill switch (Biometric or Mechanical)

# 3. The 30-Second Edge Buffer (Prevents I/O Matrix Collapse)
decision_memory = deque(maxlen=30)

def phase1_vfs_ingestion(second):
    """
    Simulates the VFS intercepting radio data and packing it into a 4KB binary sector.
    Format '6I' packs 6 unsigned integers (4 bytes each).
    """
    if second == 28:
        # Simulate a sudden Driver Cognitive Degradation / Fatigue Spike
        # Format: Tyres, Fatigue(Spikes to 88), Thermal, Hydraulics, Brakes, Fuel
        raw_data = struct.pack('6I', 45, 88, 50, 20, 30, 15)
    else:
        # Normal race wear-and-tear
        raw_data = struct.pack(
            '6I',
            random.randint(20, 60), # Tyres
            random.randint(40, 75), # Driver Fatigue
            random.randint(30, 65), # Thermal
            random.randint(10, 40), # Hydraulics
            random.randint(15, 50), # Brakes
            random.randint(5, 25)   # Fuel System
        )
    return raw_data

def phase5_surgical_strike(binary_payload):
    """
    Safely extracts the binary payload using strict struct unpacking.
    """
    extracted = struct.unpack('6I', binary_payload)
    return {
        'tyres': extracted[0],
        'driver_fatigue': extracted[1],
        'thermal': extracted[2],
        'hydraulics': extracted[3],
        'brakes': extracted[4],
        'fuel_system': extracted[5]
    }

def ibm_granite_decision_engine(telemetry):
    """
    Layer 2: Evaluates the perfectly packed data against the Dual-Trigger.
    """
    # TRIGGER 1: The Redline Veto (Absolute Single-Point Failure)
    for parameter, value in telemetry.items():
        if value >= REDLINE_VETO:
            if parameter == 'driver_fatigue':
                return True, f"BIOLOGICAL VETO: FATIGUE LIMIT CROSSSED ({value})"
            return True, f"MECHANICAL VETO: {parameter.upper()} SPIKED TO {value}"

    # TRIGGER 2: The Health Curve (Cascading Failure)
    hdi_score = sum(telemetry[k] * WEIGHTS[k] for k in WEIGHTS)

    if hdi_score >= CRITICAL_HDI_THRESHOLD:
        return True, f"CASCADING FAILURE: SYSTEM HDI HIT {round(hgi_score, 1)}"

```

```

    return False, f"SAFE (System HDI: {round(hdi_score, 1)})"

# =====
# RUNNING THE LIVE STREAM SIMULATION
# =====
print("/// INITIATING SOVEREIGN 30-SECOND EDGE BUFFER ///")
print("/// BIO-MECHANICAL SYNCHRONIZATION ONLINE ///\n")

for current_second in range(1, 35):

    # 1. Car sends data, VFS pre-packs it into Binary
    binary_chunk = phase1_vfs_ingestion(current_second)

    # 2. Phase 5 surgically extracts it with zero latency
    live_telemetry = phase5_surgical_strike(binary_chunk)
    live_telemetry['timestamp'] = current_second

    # 3. Buffer Management (The Sliding Window)
    if len(decision_memory) == 30:
        archived_data = decision_memory.popleft()
        print(f"[BACKGROUND] Sector {archived_data['timestamp']} pushed to Cloud Archive (Buffer cleared)")

    decision_memory.append(live_telemetry)

    # 4. IBM Granite evaluates the live window
    current_state = decision_memory[-1]
    pit_command, status_message = ibm_granite_decision_engine(current_state)

    # 5. Console Output for the Judges
    print(f"Sec {current_second:02d} | Buffer: {len(decision_memory):02d}/30 | {status_message}")

    if pit_command:
        print("\n" + "="*60)
        print(f"🚨 ACTION: BOX BOX BOX (Safety Override Triggered) 🚨 ")
        print(f"🚨 REASON: {status_message}")
        print("="*60 + "\n")
        break

    time.sleep(0.2)

```

```

/// INITIATING SOVEREIGN 30-SECOND EDGE BUFFER ///
/// BIO-MECHANICAL SYNCHRONIZATION ONLINE ///

```

```

Sec 01 | Buffer: 01/30 | SAFE (System HDI: 41.5)
Sec 02 | Buffer: 02/30 | SAFE (System HDI: 38.1)
Sec 03 | Buffer: 03/30 | SAFE (System HDI: 45.0)
Sec 04 | Buffer: 04/30 | SAFE (System HDI: 35.8)
Sec 05 | Buffer: 05/30 | SAFE (System HDI: 46.6)
Sec 06 | Buffer: 06/30 | SAFE (System HDI: 43.0)
Sec 07 | Buffer: 07/30 | SAFE (System HDI: 41.1)
Sec 08 | Buffer: 08/30 | SAFE (System HDI: 41.9)
Sec 09 | Buffer: 09/30 | SAFE (System HDI: 43.3)
Sec 10 | Buffer: 10/30 | SAFE (System HDI: 43.2)
Sec 11 | Buffer: 11/30 | SAFE (System HDI: 42.0)
Sec 12 | Buffer: 12/30 | SAFE (System HDI: 39.6)
Sec 13 | Buffer: 13/30 | SAFE (System HDI: 46.2)
Sec 14 | Buffer: 14/30 | SAFE (System HDI: 48.6)
Sec 15 | Buffer: 15/30 | SAFE (System HDI: 41.3)
Sec 16 | Buffer: 16/30 | SAFE (System HDI: 38.0)
Sec 17 | Buffer: 17/30 | SAFE (System HDI: 45.6)
Sec 18 | Buffer: 18/30 | SAFE (System HDI: 31.6)
Sec 19 | Buffer: 19/30 | SAFE (System HDI: 42.9)
Sec 20 | Buffer: 20/30 | SAFE (System HDI: 31.7)
Sec 21 | Buffer: 21/30 | SAFE (System HDI: 43.8)
Sec 22 | Buffer: 22/30 | SAFE (System HDI: 47.1)
Sec 23 | Buffer: 23/30 | SAFE (System HDI: 39.5)
Sec 24 | Buffer: 24/30 | SAFE (System HDI: 34.1)
Sec 25 | Buffer: 25/30 | SAFE (System HDI: 30.7)
Sec 26 | Buffer: 26/30 | SAFE (System HDI: 40.0)
Sec 27 | Buffer: 27/30 | SAFE (System HDI: 47.0)
Sec 28 | Buffer: 28/30 | BIOLOGICAL VETO: FATIGUE LIMIT CROSSSED (88)

```

```

=====
🚨 ACTION: BOX BOX BOX (Safety Override Triggered) 🚨
🚨 REASON: BIOLOGICAL VETO: FATIGUE LIMIT CROSSSED (88)
=====

```

